Annexure-I

Data Structures and Algorithm

LOVELY PROFESSIONAL UNIVERSITY

A training report

Submitted in partial fulfillment of the requirements for the award of degree of

BTECH CSE

(AI AND ML)

Submitted to

LOVELY PROFESSIONAL UNIVERSITY

PHAGWARA, PUNJAB

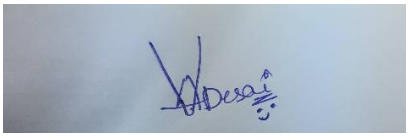
From 23/06/2025 to 30/07/2025

SUBMITTED BY

Name of student: Veeresh A

Registration Num:12319252

Signature of the student:



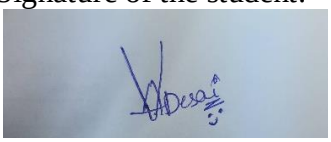
Annexure-II: Student Declaration

To whom so ever it may concern

I, **Veeresh A, 12319252,** hereby declare that the work done by me on "**Data Structures and Algorithm**" from **June-2025** to **July-2025**, is a record of original work for the partial fulfillment of the requirements for the award of the degree, **BTech Computer Science And Engineering.**

Name of the Student (Registration Number):
Veeresh A (12319252)

Signature of the student:



Dated:
August 2025

Training Certification from organization

SkillStone™
a Grazitti Interactive initiative

CERTIFICATE OF COMPLETION

CONGRATULATIONS TO

Veeresh

for successfully completing 28 days Training in **Data Structures and Algorithm** from **23rd June 2025 to 30th July 2025**. The program was aimed at providing theoretical knowledge as well as practical skills for optimum learning.

11/08/2025
Date

Alok Ramsisaria
Alok Ramsisaria (CEO)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Lovely Professional University** for offering the *"Data Structures and Algorithms"* training program, which has been an invaluable learning experience. This **28-day training program, held from 23rd June 2025 to 30th July 2025**, has provided me with a strong foundation in Data Structures and Algorithms, enhancing my problem-solving abilities and equipping me with the knowledge to implement efficient solutions in both Java and C++.

I extend my heartfelt thanks to the instructors for their expertise, guidance, and dedication. Their well-structured lessons, practical coding examples, and clear explanations have greatly improved my understanding of complex concepts.

I am equally thankful to the support staff and my fellow learners for fostering an engaging and collaborative learning environment. The projects, coding challenges, and hands-on exercises included in the course enabled me to bridge the gap between theory and practice, thereby strengthening my practical programming skills.

Finally, I am deeply appreciative of the opportunity to be a part of the Lovely Professional University learning community. Beyond technical expertise, this course has instilled in me the confidence to apply my skills effectively in future endeavours. It has been a significant milestone in my professional development, and I look forward to continuing this journey of learning and growth.

Thank you

Veeresh A
Date: August 2025

INTRODUCTION OF THE PROJECT UNDERTAKEN

➢ Objectives of the work undertaken

The primary objective of this project is to design and implement a **Train Ticket Booking System** that can automate the process of ticket reservation. In the traditional approach, booking and managing train tickets require a great deal of manual effort, which is often time-consuming and prone to errors. By developing this project in **C++**, I aimed to create a system that simplifies the reservation process and provides an efficient and user-friendly way of handling train bookings.

The key goals of the project can be summarized as follows:

- To design a system where trains can be added and managed with all necessary details, including train number, name, source, destination, total seats, and ticket price.
- To provide passengers with the ability to book tickets easily, with seats being automatically allocated in sequential order.
- To introduce a **waiting list mechanism**, where passengers are added to the list when no seats are available, and to automatically shift them to confirmed bookings once cancellations occur.
- To allow passengers to cancel tickets, thereby freeing up seats and ensuring that the system efficiently reallocates them to waiting passengers.
- To implement **file handling** so that all records of trains, bookings, and waiting lists are stored permanently and can be retrieved even after the program is closed.

Overall, the objective was to combine theoretical programming concepts with real-world applications and develop a practical system that demonstrates how **data structures, arrays, and file management** can work together in solving real-life problems

➢ Scope of the Work

The scope of this project extends across two major domains: administrative operations and passenger-level operations.

- Administrative Scope:

  Administrators or system managers can add new trains into the system with details such as train number, train name, source, destination, seating capacity, and ticket fare. They are responsible for updating the train information and ensuring that the data remains accurate for the passengers.

- Passenger_Scope:

  Passengers can interact with the system in multiple ways. They can search for trains between two locations, check availability, and book tickets. If tickets are unavailable, the system maintains a waiting list, thereby ensuring that the passenger is still given an opportunity to travel if a cancellation occurs. Additionally, passengers can cancel their tickets and view the status of their bookings by entering their PNR number.

The system uses arrays and structures for efficient storage of train and passenger information and implements queue-like logic for managing the waiting list. Furthermore, it ensures data consistency by saving the records in binary files, which makes the system useful beyond a single execution.

In future developments, the scope can be extended by incorporating a database system (MySQL, SQLite) for storing data, adding a graphical user interface (GUI) for easier interaction, and even integrating online services such as payment gateways and mobile app access.

➢ Importance and Applicability

The importance of this project lies in the fact that it directly addresses a real-world problem: train ticket reservation. Large-scale reservation systems used by Indian Railways or other organizations operate on similar principles, although they are much more complex and robust. This project acts as a scaled-down version of such a system, providing a simplified yet meaningful understanding of how reservations are handled.

From an academic perspective, the project is extremely valuable as it allows students to:

- Apply programming knowledge to develop a complete working application.
- Understand how data structures such as arrays and queues can be used in practical scenarios.
- Learn file handling concepts by storing and retrieving persistent data.
- Gain experience in designing algorithms for real-world use cases such as seat allocation, cancellation, and waiting list management.

Applicability:

- This system can serve as a prototype for small railway stations, private bus operators, or any service that requires basic reservation management.
- It can also be applied in educational environments as a learning project to help students understand the functioning of reservation systems.
- The program lays a foundation for building more complex systems by introducing essential features such as seat allocation, unique PNR generation, and cancellation handling.

Thus, the project is both practically significant and educationally valuable, combining technical concepts with a real-world scenario.

➢ Role and profile

As the developer of this project, I took complete responsibility for its **design, implementation, and testing**. My role began with analysing the requirements of a train ticket booking system and breaking them down into smaller, manageable tasks. The project involved defining clear data structures to store train and passenger details, implementing file handling for persistent storage, and applying logical steps to manage bookings, cancellations, and waiting lists.

The specific responsibilities I handled include:

- **System Design:** Creating the overall architecture of the project and deciding how trains, passengers, and bookings would be represented in the program.
- **Coding:** Writing the C++ code to implement functionalities such as adding trains, booking tickets, cancelling tickets, and maintaining the waiting list.
- **File Handling:** Using file streams to store and retrieve data, ensuring that no records were lost when the program terminated.
- **Testing and Debugging:** Running multiple test cases such as full bookings, cancellations, and waiting list reallocations to confirm that the system worked correctly under different scenarios.

By working on this project, I greatly enhanced my knowledge of **C++ programming, arrays, file handling, and structured programming techniques**. Moreover, it strengthened my problem-solving skills and gave me the confidence to design and develop complete applications from scratch.

LOVELY PROFESSIONAL UNIVERSITY

Brief description of the work done

➢ Position of Summer training and roles

During my summer training, I worked on the development of a **Train Ticket Booking** System using C++. My role was that of a trainee developer, where I was responsible for understanding the requirements, designing the structure of the application, and implementing various modules such as train management, booking, cancellation, and waiting list handling. I actively engaged in applying theoretical concepts to real-world scenarios, which helped bridge the gap between classroom learning and practical implementation.

➢ Activities/ equipment handled

The training involved several activities that were essential to the successful completion of the project:

- System Design and Planning: Preparing the flow of operations including ticket booking, cancellations, waiting list handling, and record management.
- Programming in C++: Writing and testing code to implement the core functionalities of the booking system.
- File Handling: Using file management techniques in C++ to store train details and passenger information for data persistence.
- Problem Solving: Breaking down larger tasks into smaller, manageable parts and ensuring efficiency in execution.
- Tools & Equipment: The primary equipment used was a computer system with a C++ compiler (GCC) and code editor. Additionally, text files were used to simulate data storage and retrieval.

➢ Challenges faced and how those were tackled

During the course of the training, I faced several challenges, such as:

1. Data Consistency: Ensuring that ticket details remained accurate even after multiple bookings and cancellations.
    - *Solution:* Implemented file handling carefully with validation checks to avoid data loss or duplication.
2. Waiting List Management: Allocating cancelled seats correctly to waiting passengers.
    - *Solution:* Applied queue-based logic so that passengers in the waiting list were assigned seats in order of priority.
3. Error Handling: Managing invalid inputs like incorrect train numbers or booking beyond available seats.
    - *Solution:* Designed input validation techniques and meaningful error messages to improve user experience.
4. Time Management: Balancing the project work with learning new concepts.
    - *Solution:* Followed a structured schedule and divided the work into milestones, which helped complete the project on time.

➢ Learning outcomes

The summer training proved to be a valuable learning experience. Some of the key outcomes were:

- Strengthened understanding of C++ programming concepts such as classes, objects, file handling, and data structures.
- Gained practical exposure to project development lifecycle — from requirement analysis and design to coding and testing.

- Improved problem-solving skills by tackling real-world issues like seat allocation, cancellations, and waiting list management.

- Learned the importance of data persistence and how to maintain information consistently across multiple program runs.

- Enhanced my ability to work independently and manage tasks efficiently under deadlines.

➢ Data analysis

To evaluate the project's performance, I analysed the accuracy and efficiency of the system:

- Verified that booking and cancellation operations worked correctly in different scenarios.
- Checked whether waiting list passengers were properly allocated seats once they became available.
- Ensured that records were stored and retrieved correctly from files after closing and reopening the program.
- Measured response time for operations, which was quick due to the use of arrays and simple data structures.
- Conducted multiple test cases to validate the system, ensuring reliability and correctness of outputs.

The data analysis confirmed that the system successfully automated the ticket booking process and handled all operations as expected.

- **Project Details**

Technology Used

1. Programming Language – C++

   The project was developed using C++, as it provides fast execution, simplicity, and flexibility in handling user inputs and outputs. Its structured programming approach made it easier to implement ticket booking operations such as reservation, cancellation, and displaying passenger records.

2. Data Structures – Arrays and Functions

   To manage passenger records, train details, and ticket availability, arrays were used as the primary data structure. Along with this, functions were implemented to modularize different operations like booking tickets, checking availability, and displaying status, which made the program easier to understand and maintain.

3. File Handling – Storage of Data

   File handling in C++ was used to store passenger details, booking records, and train schedules. This allowed the application to maintain data persistence, so that the records could be retrieved even after the program was closed.

4. Integrated Development Environment (IDE) – Visual Studio

   The program was written, compiled, and debugged using Visual Studio, which provided powerful features like an in-built debugger, user-friendly interface, and smooth execution of the C++ project.

5. Operating System – Windows

   The project was designed, implemented, and tested on the Windows operating system, which provided a stable environment for program execution and file handling operations.

CODE:

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

struct Train {
    int trainNo;
    char name[50];
    char source[30];
    char destination[30];
    int totalSeats;
    int availableSeats;
    float price;
};

struct Passenger {
    int pnr;
    char name[50];
    int age;
    int trainNo;
    int seatNo;
};

Train trains[20];
Passenger bookings[200];
Passenger waitingList[50]; // queue for waiting passengers
int trainCount = 0, bookingCount = 0, waitingCount = 0;
int pnrCounter = 1000;

// Save trains and bookings to files
void saveData() {
    ofstream fout1("trains.dat", ios::binary);
    fout1.write((char*)&trainCount, sizeof(trainCount));
    fout1.write((char*)trains, sizeof(Train) * trainCount);
    fout1.close();

    ofstream fout2("bookings.dat", ios::binary);
    fout2.write((char*)&bookingCount, sizeof(bookingCount));
    fout2.write((char*)bookings, sizeof(Passenger) * bookingCount);
    fout2.close();
}

// Load trains and bookings from files
void loadData() {
    ifstream fin1("trains.dat", ios::binary);
    if(fin1) {
        fin1.read((char*)&trainCount, sizeof(trainCount));
```

```
        fin1.read((char*)trains, sizeof(Train) * trainCount);
        fin1.close();
    }
    ifstream fin2("bookings.dat", ios::binary);
    if(fin2) {
        fin2.read((char*)&bookingCount, sizeof(bookingCount));
        fin2.read((char*)bookings, sizeof(Passenger) * bookingCount);
        fin2.close();
    }
}

void addTrain() {
    cout << "Enter Train Number: ";
    cin >> trains[trainCount].trainNo;
    cout << "Enter Train Name: ";
    cin.ignore();
    cin.getline(trains[trainCount].name, 50);
    cout << "Enter Source: ";
    cin.getline(trains[trainCount].source, 30);
    cout << "Enter Destination: ";
    cin.getline(trains[trainCount].destination, 30);
    cout << "Enter Total Seats: ";
    cin >> trains[trainCount].totalSeats;
    trains[trainCount].availableSeats = trains[trainCount].totalSeats;
    cout << "Enter Ticket Price: ";
    cin >> trains[trainCount].price;

    trainCount++;
    saveData();
    cout << "✅ Train Added Successfully!\n";
}

void viewTrains() {
    cout << "\nAvailable Trains:\n";
    for(int i=0;i<trainCount;i++) {
        cout << trains[i].trainNo << " - " << trains[i].name
             << " (" << trains[i].source << " -> " << trains[i].destination << ")"
             << " Seats: " << trains[i].availableSeats
             << " Price: " << trains[i].price << "\n";
    }
}

void searchTrain() {
    char src[30], dest[30];
    cout << "Enter Source: ";
    cin.ignore();
    cin.getline(src, 30);
    cout << "Enter Destination: ";
    cin.getline(dest, 30);

    bool found = false;
```

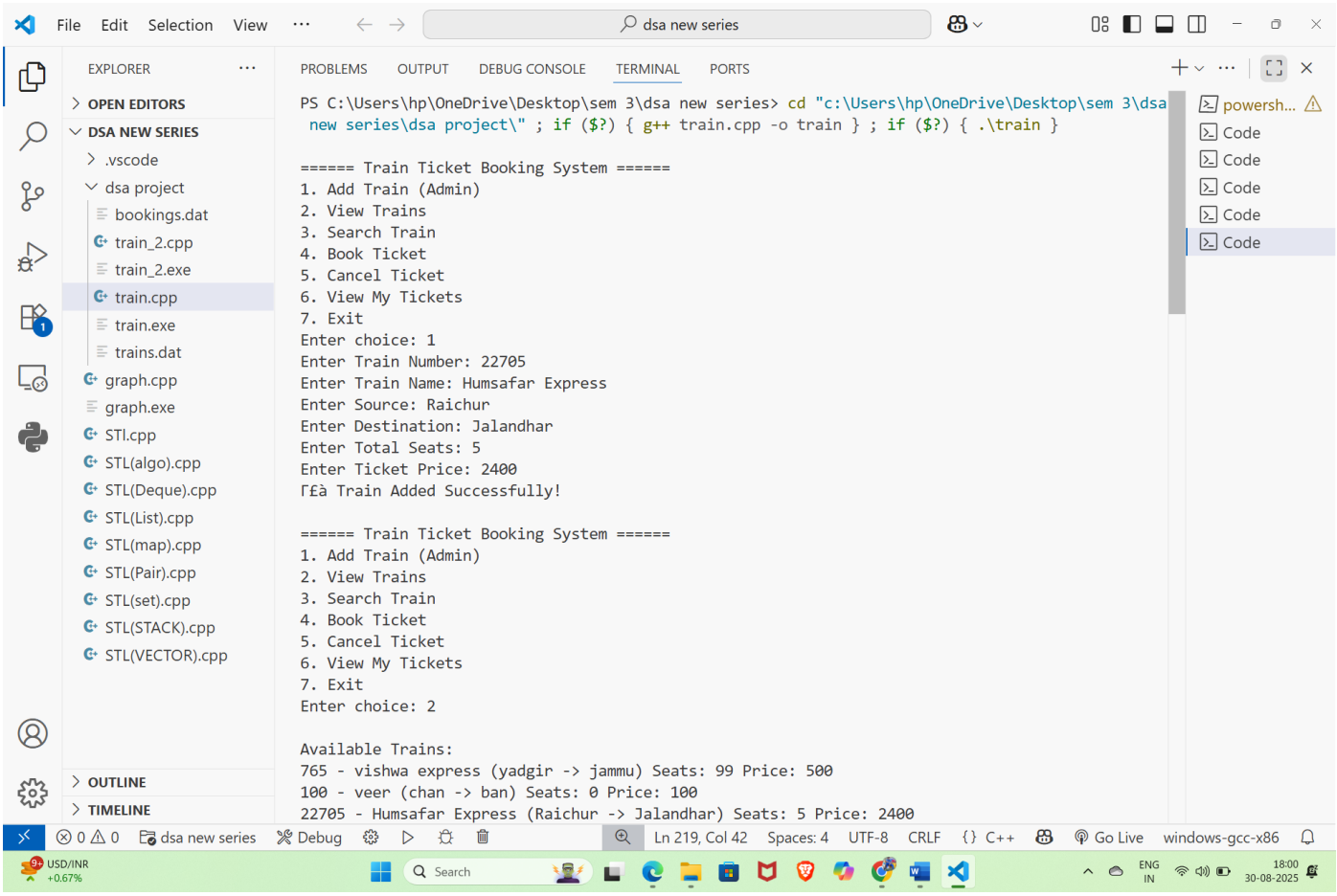
```
    for(int i=0;i<trainCount;i++) {
        if(strcmp(trains[i].source, src) == 0 && strcmp(trains[i].destination, dest) == 0) {
            cout << trains[i].trainNo << " - " << trains[i].name
                 << " Seats: " << trains[i].availableSeats
                 << " Price: " << trains[i].price << "\n";
            found = true;
        }
    }
    if(!found) cout << "No trains found!\n";
}

void bookTicket() {
    int tno;
    cout << "Enter Train Number to Book: ";
    cin >> tno;
    for(int i=0;i<trainCount;i++) {
        if(trains[i].trainNo == tno) {
            if(trains[i].availableSeats > 0) {
                Passenger p;
                p.pnr = pnrCounter++;
                p.trainNo = tno;
                cout << "Enter Passenger Name: ";
                cin.ignore();
                cin.getline(p.name,50);
                cout << "Enter Age: ";
                cin >> p.age;
                p.seatNo = trains[i].totalSeats - trains[i].availableSeats + 1;

                bookings[bookingCount++] = p;
                trains[i].availableSeats--;

                saveData();
                cout << "🎟 Ticket Booked! PNR: " << p.pnr << ", Seat No: " << p.seatNo << "\n";
                return;
            } else {
                // Add to waiting list
                Passenger wp;
                wp.pnr = pnrCounter++;
                wp.trainNo = tno;
                cout << "Enter Passenger Name (WAITING LIST): ";
                cin.ignore();
                cin.getline(wp.name,50);
                cout << "Enter Age: ";
                cin >> wp.age;
                wp.seatNo = -1; // not assigned yet
                waitingList[waitingCount++] = wp;
                cout << "⚠ No seats available! Added to waiting list. PNR: " << wp.pnr << "\n";
                return;
            }
        }
```

```
    }
    cout << "Train not found!\n";
}

void cancelTicket() {
    int pnr;
    cout << "Enter PNR to Cancel: ";
    cin >> pnr;
    for(int i=0;i<bookingCount;i++) {
        if(bookings[i].pnr == pnr) {
            int tno = bookings[i].trainNo;
            cout << "❌ Ticket Cancelled for " << bookings[i].name << "\n";
            // shift bookings array
            for(int j=i;j<bookingCount-1;j++) bookings[j] = bookings[j+1];
            bookingCount--;

            // free seat
            for(int k=0;k<trainCount;k++) {
                if(trains[k].trainNo == tno) {
                    trains[k].availableSeats++;
                    // if waiting list not empty → give seat
                    if(waitingCount > 0) {
                        Passenger wp = waitingList[0];
                        for(int w=0;w<waitingCount-1;w++) waitingList[w] = waitingList[w+1];
                        waitingCount--;

                        wp.seatNo = trains[k].totalSeats - trains[k].availableSeats + 1;
                        bookings[bookingCount++] = wp;
                        trains[k].availableSeats--;
                        cout << "✅ Seat given to waiting passenger: " << wp.name << " (PNR: " << wp.pnr <<
")\n";
                    }
                }
            }
            saveData();
            return;
        }
    }
    cout << "PNR not found!\n";
}

void viewMyTickets() {
    int pnr;
    cout << "Enter PNR to View Ticket: ";
    cin >> pnr;
    for(int i=0;i<bookingCount;i++) {
        if(bookings[i].pnr == pnr) {
            cout << "🎟 Ticket Found!\n";
            cout << "PNR: " << bookings[i].pnr
                 << " Name: " << bookings[i].name
```

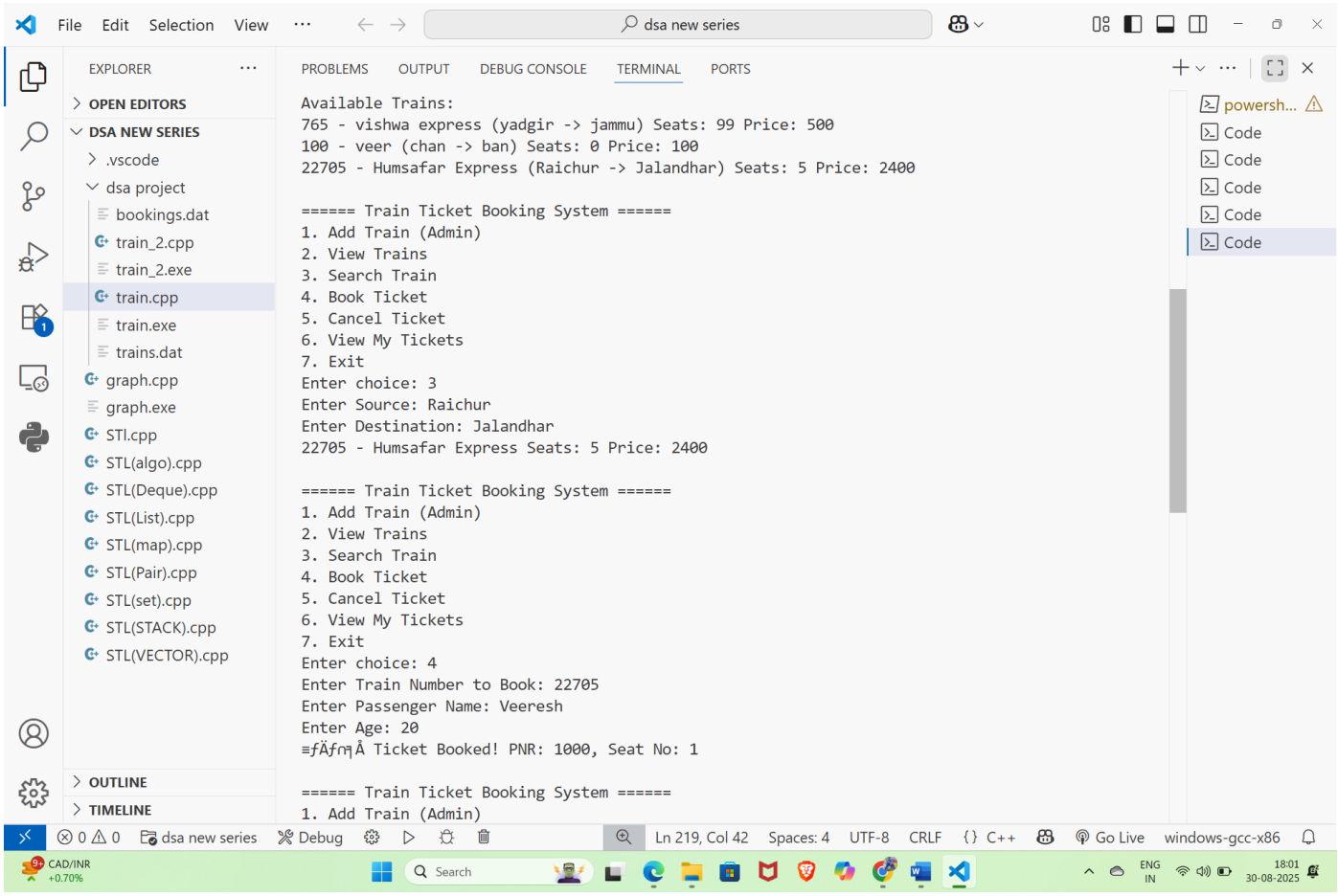
```
            << " Age: " << bookings[i].age
            << " Train No: " << bookings[i].trainNo
            << " Seat: " << bookings[i].seatNo << "\n";
        return;
      }
    }
    cout << "No ticket found for given PNR!\n";
}

int main() {
    loadData();
    int choice;
    while(true) {
        cout << "\n====== Train Ticket Booking System ======\n";
        cout << "1. Add Train (Admin)\n2. View Trains\n3. Search Train\n";
        cout << "4. Book Ticket\n5. Cancel Ticket\n6. View My Tickets\n7. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
        switch(choice) {
            case 1: addTrain(); break;
            case 2: viewTrains(); break;
            case 3: searchTrain(); break;
            case 4: bookTicket(); break;
            case 5: cancelTicket(); break;
            case 6: viewMyTickets(); break;
            case 7: saveData(); return 0;
            default: cout << "Invalid choice!\n";
        }
    }
}
```

OUTPUT:

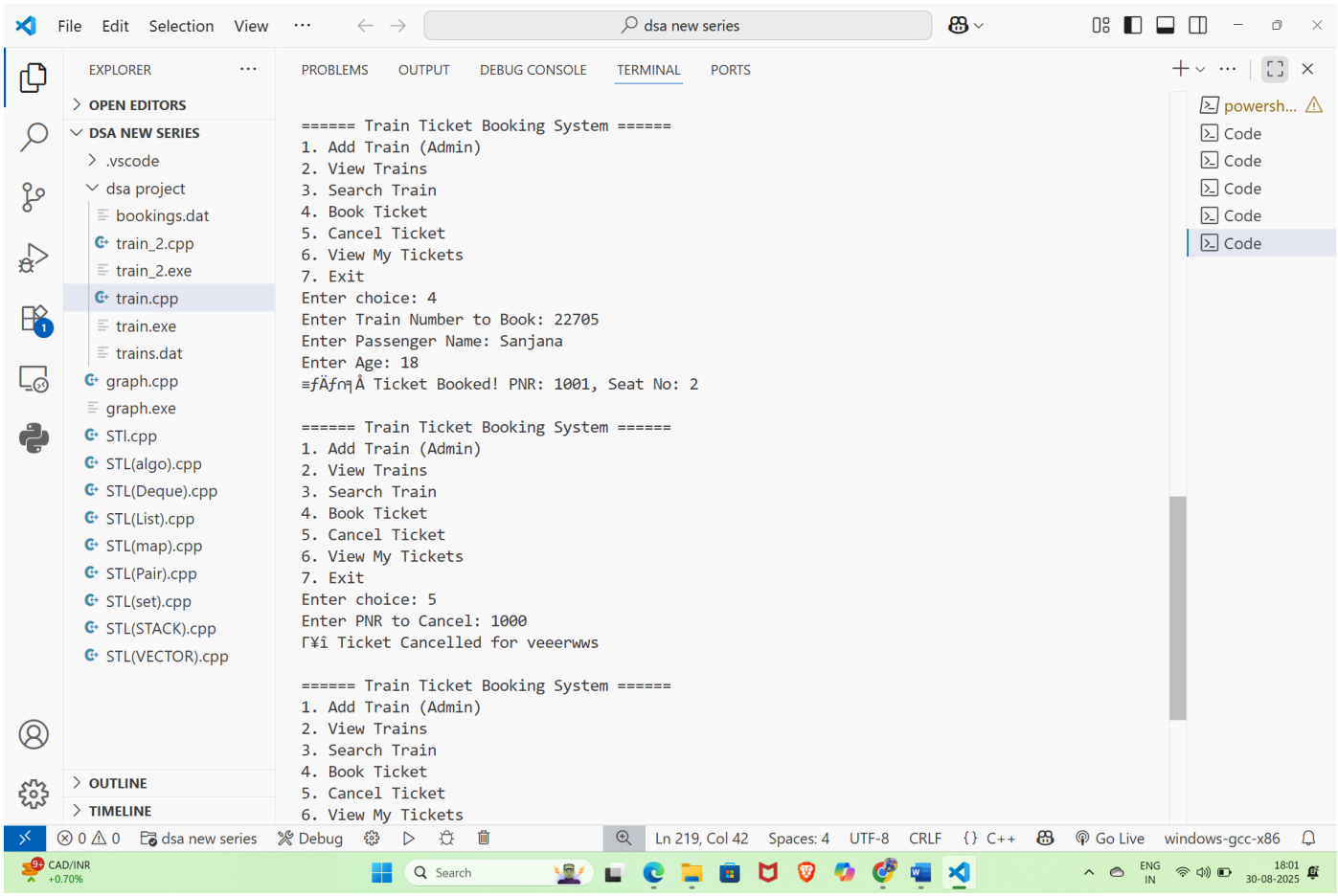
1.


File Edit Selection View ···
dsa new series
EXPLORER
OPEN EDITORS
DSA NEW SERIES
.vscode
dsa project
bookings.dat
train_2.cpp
train_2.exe
train.cpp
train.exe
trains.dat
graph.cpp
graph.exe
STl.cpp
STL(algo).cpp
STL(Deque).cpp
STL(List).cpp
STL(map).cpp
STL(Pair).cpp
STL(set).cpp
STL(STACK).cpp
STL(VECTOR).cpp
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\hp\OneDrive\Desktop\sem 3\dsa new series> cd "c:\Users\hp\OneDrive\Desktop\sem 3\dsa new series\dsa project\" ; if ($?) { g++ train.cpp -o train } ; if ($?) { .\train }
====== Train Ticket Booking System ======
1. Add Train (Admin)
2. View Trains
3. Search Train
4. Book Ticket
5. Cancel Ticket
6. View My Tickets
7. Exit
Enter choice: 1
Enter Train Number: 22705
Enter Train Name: Humsafar Express
Enter Source: Raichur
Enter Destination: Jalandhar
Enter Total Seats: 5
Enter Ticket Price: 2400
Γ£à Train Added Successfully!
====== Train Ticket Booking System ======
1. Add Train (Admin)
2. View Trains
3. Search Train
4. Book Ticket
5. Cancel Ticket
6. View My Tickets
7. Exit
Enter choice: 2
Available Trains:
765 - vishwa express (yadgir -> jammu) Seats: 99 Price: 500
100 - veer (chan -> ban) Seats: 0 Price: 100
22705 - Humsafar Express (Raichur -> Jalandhar) Seats: 5 Price: 2400
powersh...
Code
Code
Code
Code
Code
Ln 219, Col 42 Spaces: 4 UTF-8 CRLF C++ Go Live windows-gcc-x86
USD/INR +0.67%
Search
ENG IN
18:00 30-08-2025

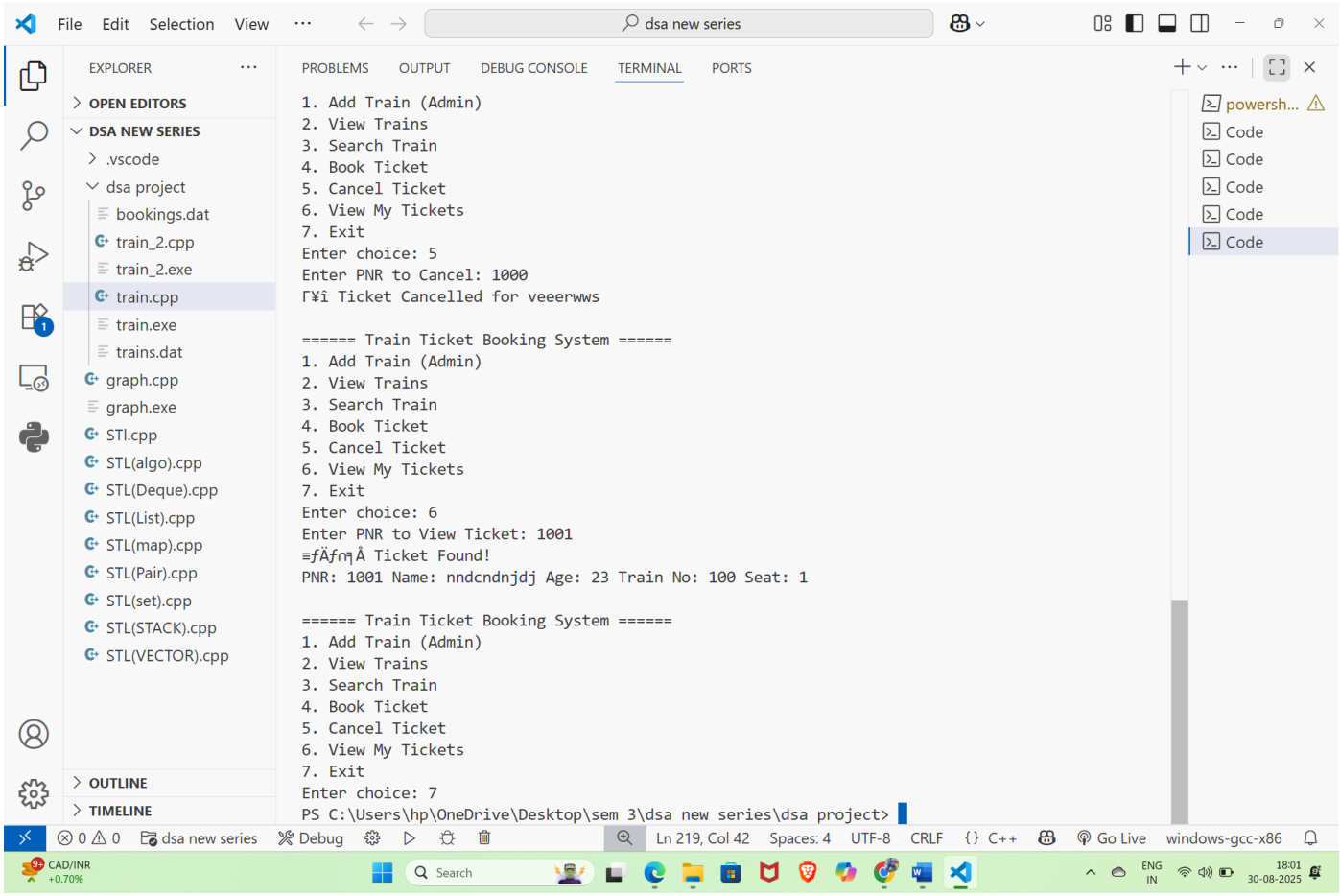
2.


File Edit Selection View ···
dsa new series
EXPLORER
OPEN EDITORS
DSA NEW SERIES
.vscode
dsa project
bookings.dat
train_2.cpp
train_2.exe
train.cpp
train.exe
trains.dat
graph.cpp
graph.exe
STl.cpp
STL(algo).cpp
STL(Deque).cpp
STL(List).cpp
STL(map).cpp
STL(Pair).cpp
STL(set).cpp
STL(STACK).cpp
STL(VECTOR).cpp
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Available Trains:
765 - vishwa express (yadgir -> jammu) Seats: 99 Price: 500
100 - veer (chan -> ban) Seats: 0 Price: 100
22705 - Humsafar Express (Raichur -> Jalandhar) Seats: 5 Price: 2400
====== Train Ticket Booking System ======
1. Add Train (Admin)
2. View Trains
3. Search Train
4. Book Ticket
5. Cancel Ticket
6. View My Tickets
7. Exit
Enter choice: 3
Enter Source: Raichur
Enter Destination: Jalandhar
22705 - Humsafar Express Seats: 5 Price: 2400
====== Train Ticket Booking System ======
1. Add Train (Admin)
2. View Trains
3. Search Train
4. Book Ticket
5. Cancel Ticket
6. View My Tickets
7. Exit
Enter choice: 4
Enter Train Number to Book: 22705
Enter Passenger Name: Veeresh
Enter Age: 20
≡ƒÄƒ┬Â Ticket Booked! PNR: 1000, Seat No: 1
====== Train Ticket Booking System ======
1. Add Train (Admin)
powersh...
Code
Code
Code
Code
Code
Ln 219, Col 42 Spaces: 4 UTF-8 CRLF C++ Go Live windows-gcc-x86
CAD/INR +0.70%
Search
ENG IN
18:01 30-08-2025

3.


File Edit Selection View ···
dsa new series
EXPLORER
OPEN EDITORS
DSA NEW SERIES
.vscode
dsa project
bookings.dat
train_2.cpp
train_2.exe
train.cpp
train.exe
trains.dat
graph.cpp
graph.exe
STl.cpp
STL(algo).cpp
STL(Deque).cpp
STL(List).cpp
STL(map).cpp
STL(Pair).cpp
STL(set).cpp
STL(STACK).cpp
STL(VECTOR).cpp
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powersh...
Code
Code
Code
Code
Code
====== Train Ticket Booking System ======
1. Add Train (Admin)
2. View Trains
3. Search Train
4. Book Ticket
5. Cancel Ticket
6. View My Tickets
7. Exit
Enter choice: 4
Enter Train Number to Book: 22705
Enter Passenger Name: Sanjana
Enter Age: 18
≡ƒÄƒ¬Å Ticket Booked! PNR: 1001, Seat No: 2
====== Train Ticket Booking System ======
1. Add Train (Admin)
2. View Trains
3. Search Train
4. Book Ticket
5. Cancel Ticket
6. View My Tickets
7. Exit
Enter choice: 5
Enter PNR to Cancel: 1000
Γ¥î Ticket Cancelled for veeerwws
====== Train Ticket Booking System ======
1. Add Train (Admin)
2. View Trains
3. Search Train
4. Book Ticket
5. Cancel Ticket
6. View My Tickets
0 0 dsa new series Debug
Ln 219, Col 42 Spaces: 4 UTF-8 CRLF C++ Go Live windows-gcc-x86
CAD/INR +0.70%
Search
ENG IN
18:01 30-08-2025

4.


File Edit Selection View ···
dsa new series
EXPLORER
OPEN EDITORS
DSA NEW SERIES
.vscode
dsa project
bookings.dat
train_2.cpp
train_2.exe
train.cpp
train.exe
trains.dat
graph.cpp
graph.exe
STl.cpp
STL(algo).cpp
STL(Deque).cpp
STL(List).cpp
STL(map).cpp
STL(Pair).cpp
STL(set).cpp
STL(STACK).cpp
STL(VECTOR).cpp
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
1. Add Train (Admin)
2. View Trains
3. Search Train
4. Book Ticket
5. Cancel Ticket
6. View My Tickets
7. Exit
Enter choice: 5
Enter PNR to Cancel: 1000
Γ¥î Ticket Cancelled for veeerwws
====== Train Ticket Booking System ======
1. Add Train (Admin)
2. View Trains
3. Search Train
4. Book Ticket
5. Cancel Ticket
6. View My Tickets
7. Exit
Enter choice: 6
Enter PNR to View Ticket: 1001
≡ƒÄƒ¬Å Ticket Found!
PNR: 1001 Name: nndcndnjdj Age: 23 Train No: 100 Seat: 1
====== Train Ticket Booking System ======
1. Add Train (Admin)
2. View Trains
3. Search Train
4. Book Ticket
5. Cancel Ticket
6. View My Tickets
7. Exit
Enter choice: 7
PS C:\Users\hp\OneDrive\Desktop\sem 3\dsa new series\dsa project>
powersh...
Code
Code
Code
Code
Code
dsa new series
Debug
Ln 219, Col 42
Spaces: 4
UTF-8
CRLF
C++
Go Live
windows-gcc-x86
CAD/INR +0.70%
Search
ENG IN
18:01 30-08-2025

OUTCOME:

The Train Ticket Booking System project successfully met its primary objective of creating a simple, efficient, and user-friendly console-based application for handling ticket reservations. By making use of arrays, functions, and file handling in C++, the project was able to replicate the basic operations of a real-world booking system such as ticket reservation, cancellation, checking seat availability, and displaying passenger details.

The outcome of the project can be summarized as follows:

1. Automation of Booking Process

   The project reduced manual effort by automating the ticket reservation process, ensuring faster and error-free operations.
2. Efficient Data Management

   Through file handling, passenger records and booking details could be stored and retrieved easily, ensuring data persistence even after program termination.
3. Practical Implementation of C++ Concepts

   The project gave hands-on experience with fundamental programming concepts like arrays, modular programming using functions, and file handling. It also strengthened problem-solving and logical thinking skills.
4. User-Friendly Interaction

   A clear console menu was implemented, allowing users to interact with the system easily and perform required operations step by step.
5. Learning Outcome

   The project helped in understanding how real-world problems can be solved using programming. It also provided insights into how ticket booking systems work in practice, making the learning more application-oriented.

CONCLUSION

The development of the **Train Ticket Booking System** has been a valuable learning experience and a practical application of the concepts of **C++ programming and data structures**. The project successfully simulated the essential features of a reservation system such as booking tickets, canceling reservations, checking availability, and maintaining passenger details using simple yet effective logic.

Through this project, the importance of **queues, arrays, functions, and file handling** in solving real-world problems was understood clearly. The implementation not only enhanced programming skills but also improved the ability to design structured solutions to common problems. The system, though basic, demonstrates how a computerized booking process can save time, reduce errors, and improve efficiency compared to manual methods.

While the project fulfills its objectives, there is also scope for enhancement. Features such as **payment integration, seat preference options, and a graphical interface** could be added to make the system more realistic and user-friendly.

In conclusion, the project served as a strong foundation in both **technical learning and practical application**, bridging the gap between classroom knowledge and real-world requirements. It highlighted the significance of logical thinking, problem-solving, and the role of programming in building useful applications for society.

REFERENCES

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Malik, D. S. (2010). Data Structures Using C++. Cengage Learning.

GeeksforGeeks – Data Structures and Algorithms Tutorials. Retrieved from: https://www.geeksforgeeks.org.

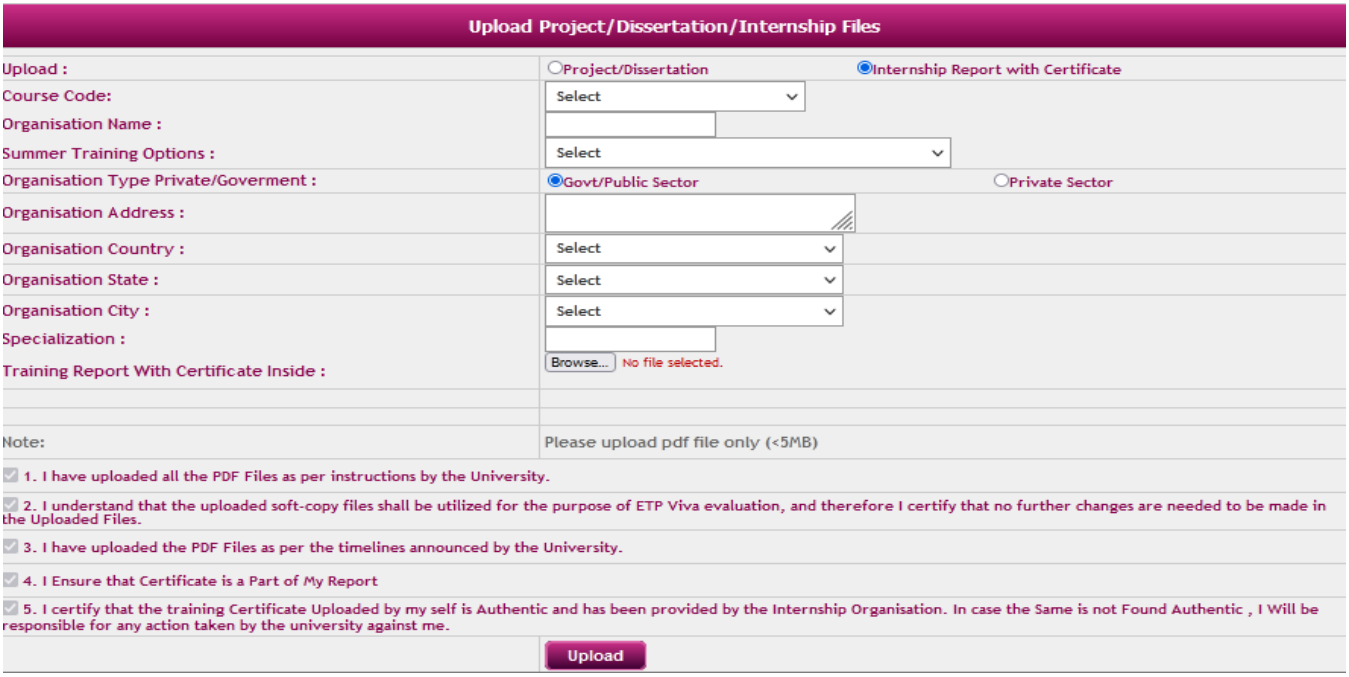
HackerRank – Data Structures and Algorithms Practice Problems.
Retrieved from: https://www.hackerrank.com/

Leetcode – Programming Challenges and Interview Preparation.
Retrieved from: https://leetcode.com/

Skillstone- Practiced assignment given from skillstone
Retrieved from: https://www.skillstone.in/account/

Instructions for Uploading Training Certificate/Report

Step 1: Go To UMS Navigation-->LMS-->Upload Research Project/Internship certificate Details> Select Upload Research Project/Internship Certificate--> Select Internship Report with Certificate option as shown below



Step 2: Select Course Code >Fill/Choose Relevant Details> Upload File (less than 5MB) > Read Notes> Click Upload

NOTE:

1) Upload Final Report with **Training Certificate** inside the report **(less than 5 MB) for all 3 options**